

Final Project Report

Group # 5 - Credit Card Fraud Detection

Dhwaj Jain — Maanya Raichura

DTSC301 • Machine Learning for Data Science-1 • December 13, 2025

Abstract

This project focuses on detecting fraudulent credit card transactions using machine learning techniques, addressing a highly imbalanced classification problem with significant real-world importance for banks, customers, and financial institutions. We use the publicly available Kaggle dataset “Credit Card Fraud Detection” (<https://www.kaggle.com/datasets/kartik2112/fraud-detection>), containing over 11 million transactions from 2019–2020. After performing preprocessing and exploratory analysis, we trained and evaluated multiple models including Logistic Regression, Decision Trees, Random Forest, XGBoost, LightGBM, and Support Vector Machines. Model performance was assessed using Accuracy, Precision, Recall, F1 Score, ROC-AUC, and PR-AUC, with F1 and PR-AUC emphasized due to severe class imbalance. Our results show that the tuned Decision Tree model achieved the strongest balance between precision and recall ($F1 = 0.7476$, $PR-AUC = 0.7805$), while boosting models such as XGBoost and LightGBM also demonstrated high discriminative power. Overall, the study highlights the importance of feature engineering, careful model tuning, and selecting metrics appropriate for rare-event detection, providing practical insights for deploying fraud detection systems in real financial settings.

1 Introduction and Problem Statement

The main objective of this project is to detect fraudulent credit card transactions with high accuracy while minimizing false negatives, as missing a fraud case is significantly more costly than flagging a legitimate transaction. The primary stakeholders include banks, credit card customers, and financial service providers, all of whom aim to reduce financial losses and ensure secure transaction processing.

To evaluate model performance, we use the following metrics: **Accuracy**, **Precision**, **Recall**, **F1 Score**, **ROC-AUC**, and **PR-AUC**. Among these, our primary evaluation metric is the **F1 Score**, since the dataset is highly imbalanced and it is crucial to maximise the correct identification of fraud cases without excessively increasing false positives. Our target performance threshold is an **F1 Score of at least 0.75**.

We assume that the features provided in the dataset are sufficient for predicting fraud, that the preprocessing steps adequately handle inconsistencies, and that the data distribution reasonably reflects real-world transaction behaviour. However, the dataset also presents certain constraints: the number of fraud cases is extremely low, the data is from **2019-2020** and may not fully represent current fraud patterns, and some features may contain noise or inaccuracies.

2 Dataset and Data Access

2.1 Data Overview

We have used the Credit Card Transactions Fraud Detection Dataset dataset that we got from Kaggle, and the data was already separated into two separate files: `fraudTrain1.csv` and `fraudTest.csv`. These files contain transactional data recorded between January 2019 and December 2020. The training set consists of approximately **1.3 million transactions**, while the test set contains around **550,000 transactions**. The target variable, `is_fraud`, indicates whether a transaction is **fraudulent (1)** or **legitimate (0)**. The feature set includes numerical

variables such as transaction amount, customer and merchant coordinates, city population, and temporal information, along with categorical variables such as transaction category and customer gender.

The dataset is **highly imbalanced**, with fraudulent transactions comprising only about 0.5728% of all observations, reflecting the real-world fraud detection challenges. The original data source is publicly accessible at:

<https://www.kaggle.com/datasets/kartik2112/fraud-detection/code>.

To maintain evaluation consistency, we used the predefined train-test split provided through the two files rather than performing an additional random split.

2.2 Data Preprocessing

2.2.1 Dimensional Reduction

During preprocessing, we first removed columns containing identifiers and personal information, such as customer names, card numbers, merchant names, job titles, and address fields. These features do not contribute to the prediction of fraud and may also cause overfitting because they uniquely identify individuals rather than capture useful patterns.

2.2.2 Temporal preprocessing

Next, we converted the transaction timestamp (`trans_date_trans_time`) to a proper datetime format, and we extracted three informative temporal features: `hour`, `day` and `month`. These help reveal when fraudulent activity is more likely to occur. After extracting these components, the original timestamp column was removed.

2.2.3 Data Encoding

Categorical variables such as `category` and `gender` were transformed using label encoding to convert them into numerical form suitable for machine learning algorithms.

This preprocessing strategy ensured that the dataset was both analytically sound and aligned with the requirements of fraud detection models, enabling fair comparison across all the models.

3 Exploratory Data Analysis (EDA)

Before model training, a rigorous Exploratory Data Analysis (EDA) was conducted to understand the data distribution, assess data quality, and identify discriminative patterns between fraudulent and legitimate transactions.

Target Distribution: The most defining characteristic of this dataset is its extreme class imbalance. As visualised in Figure 1, the target variable `is_fraud` is heavily skewed.

- **Fraud Rate:** Approximately **0.5728%** (Insert value from your code output).
- **Implication:** This severe imbalance necessitates the use of precision-recall metrics over accuracy and suggests that random sampling would likely yield a dataset with insufficient positive samples for training deep learning models without adjustment.

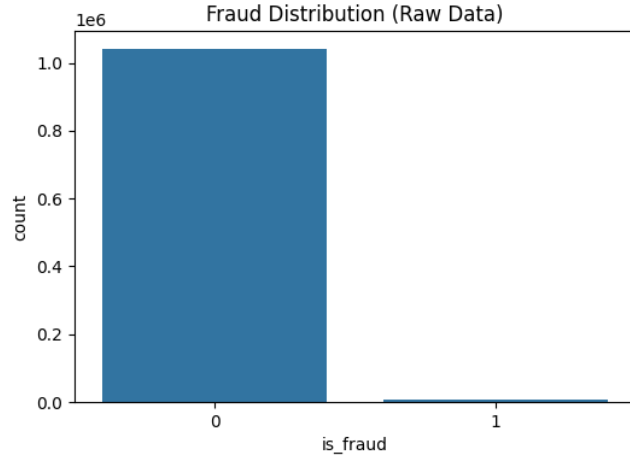


Figure 1: Target Distribution. The dataset is overwhelmingly dominated by the majority class (0), necessitating specialized handling techniques.

3.1 Temporal Analysis

Fraud Rate by Hour Analysing the fraud probability against the time of day revealed a strong cyclical pattern. As shown in Figure 2, fraudulent transactions do not occur uniformly.

- **Observation:** Fraud rates often spike during "off-hours" (e.g., late night or early morning) when legitimate customer activity is low, making anomalies more statistically prominent.
- **Decision:** This justifies the inclusion of `hour` as a key feature for the decision tree and XGBoost models.

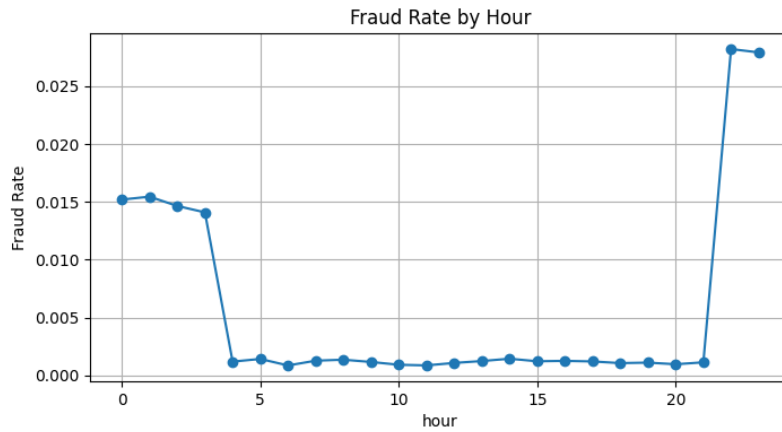


Figure 2: Fraud Rate by Hour. The non-linear relationship confirms that time-of-day is a critical predictor.

3.2 Multivariate Analysis

3.2.1 Feature Correlations (Spearman)

Given that financial data (like transaction amounts) is rarely normally distributed, we utilised **Spearman's Rank Correlation** rather than Pearson correlation to detect monotonic relationships. The heatmap (Figure 3) highlights the top 10 features correlated with `is_fraud`.

- **Transaction Amount (`amt`):** showed the strongest positive correlation with fraud.

High-value transactions are inherently riskier.

- **Demographics:** Features such as `city_pop` showed negligible correlation, suggesting that fraud is not strictly tied to the size of the cardholder's location.

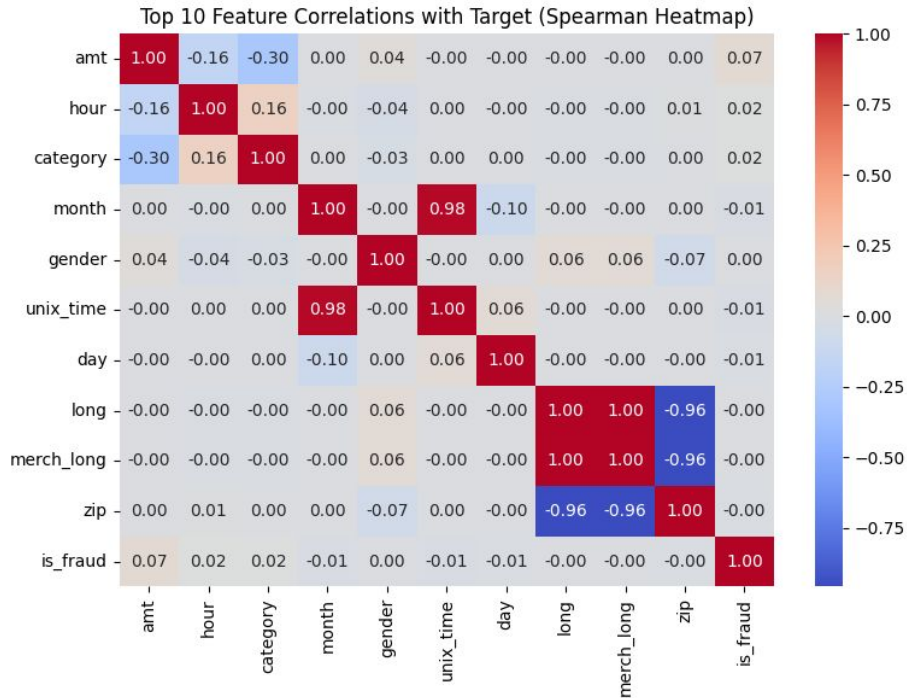


Figure 3: Spearman Correlation Heatmap (Top 10 Features). Transaction amount acts as the primary driver for fraud probability.

3.3 Impact of Oversampling: The SMOTE Distortion

A critical part of our Advanced EDA was an experiment to determine the viability of Synthetic Minority Over-sampling Technique (SMOTE). We compared the feature correlations of the original (No-SMOTE) dataset against a SMOTE-resampled version.

Figure 4 illustrates the **Correlation Shift**.

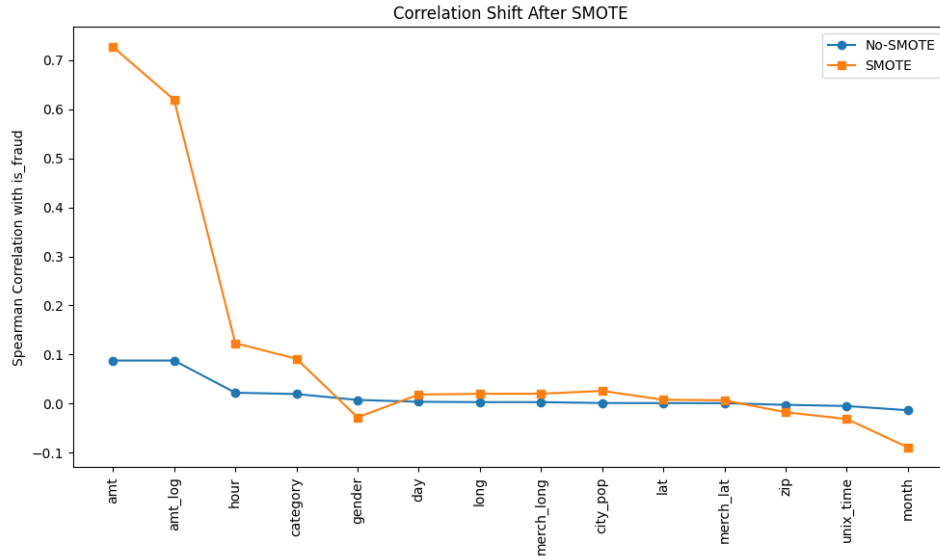


Figure 4: Correlation Shift: Original Data vs. SMOTE Data. The orange line (SMOTE) shows drastically inflated correlations compared to the blue line (Original).

Key Observation: In the original data (Blue line), the correlation between `amt` and `is_fraud` is moderate. However, after applying SMOTE (Orange line), this correlation jumps to near 0.7 - 0.8.

Interpretation: SMOTE generates synthetic points by interpolating between existing fraud cases. Since many fraud cases in the training set happened to have high amounts, SMOTE blindly created thousands of new "high-amount" fraud points.

Conclusion: Based on this EDA finding, **we decided not to use SMOTE for the final model training**, opting instead for Cost-Sensitive learning (Class Weights) to preserve the true data distribution.

4 Code and Implementation

4.1 Implementation and Reproducibility Guide

4.1.1 Repository Structure

```
.
main.py
report.pdf
README.txt
```

4.1.2 Environment Setup

Operating System The codebase is compatible with:

- Windows 10 / 11
- macOS (Intel or Apple Silicon)
- Linux (Ubuntu 20.04 or later)

Python Version Python version 3.10 or higher is required. The pipeline was tested on Python 3.13.2 and Python 3.11.9.

To verify the Python installation:

```
python --version
```

If Python is not installed, it can be downloaded from:

<https://www.python.org/downloads/>

Windows users must ensure that Python is added to the system PATH during installation.

Recommended IDE

The pipeline can be executed from any Python-compatible environment. Visual Studio Code (VS Code) is recommended.

Recommended VS Code extensions:

- Python (Microsoft)
- Pylance
- Jupyter (for inline plotting)

Virtual Environment (Recommended)

1. From the project directory, create a virtual environment:

```
python -m venv venv
```

2. Activate the environment:

Windows:

```
venv\Scripts\activate
```

macOS / Linux:

```
source venv/bin/activate
```

After activation, the terminal should display (venv).

Required Packages

Install all dependencies using:

```
pip install pandas numpy matplotlib seaborn scikit-learn imbalanced-learn  
          xgboost scipy shap lime lightgbm
```

To verify installation:

```
pip list
```

4.1.3 Dataset Description and Access

Dataset Summary

The dataset consists of credit card transaction records with a binary fraud label (`is_fraud`). Fraud cases represent 0.57% of all transactions, making this a highly imbalanced classification problem.

Key feature groups include:

- Transaction amount and merchant category

- Temporal features (hour, day, month)
- Location-related variables
- Binary fraud indicator

Dataset Location

The dataset is hosted on a shared drive:

<https://drive.google.com/drive/folders/1g0nJlaGWAQW-Jr1TrljRvXV5ynh9w-cw>

Usage instructions:

1. Download `fraudTrain1.csv` and `fraudTest.csv`
2. Place both files in the same directory as `main.py`
3. Do not rename the files

4.1.4 How to Run the Code

From the project directory, execute:

```
python main.py
```

The script executes sequentially:

1. Data loading
2. Basic EDA (raw data)
3. Preprocessing
4. Advanced EDA (post-cleaning)
5. Model training and hyperparameter tuning
6. Evaluation and visualisation
7. Final performance comparison

Execution time may be several minutes due to dataset size and tuning procedures.

4.1.5 Reproducibility

To ensure reproducibility:

- Fixed random seeds were used throughout (`random_state = 42`)
- `RandomizedSearchCV` was executed with a fixed seed
- Subsampling for tuning was deterministic
- All metrics were evaluated on a fixed hold-out test set

Hardware assumptions:

- CPU-based execution (no GPU dependency)
- Recommended minimum: 16 GB RAM, quad-core CPU

Due to stochastic algorithms and randomized hyperparameter search, minor metric fluctuations across runs are expected, though relative model rankings remain stable.

4.2 Implementation Notes and Changes from Interim Project

During development, a few changes from the interim project were made to improve efficiency and avoid performance bottlenecks:

- **Hyperparameter Tuning Scope:** Due to the extremely large dataset, tuning was performed on a subsample of 120,000 rows for most models and 50,000 rows for SVM to reduce runtime and memory load.
- **Removal of SMOTE**
- **SVM Scaling:** As SVM with an RBF kernel is computationally expensive, we trained it using a scaled subset rather than the full dataset.
- **Class Imbalance Handling:** Instead of training on a SMOTE-oversampled dataset, SMOTE was used only for correlation comparison and exploratory analysis to avoid synthetic data affecting model generalisation.
- **SHAP and LIME Restrictions:** Interpretability techniques were applied only to XGBoost because tree-based SHAP is computationally scalable and performs reliably on structured tabular data.
- **Feature Reduction**

Overall, the final implementation balances computational feasibility with model accuracy, interpretability, and robustness.

5 Experiments and Results

5.1 Evaluation Metrics

In credit card fraud detection, the dataset is highly imbalanced, with fraudulent transactions typically representing less than 0.5% of all activity. For this reason, traditional metrics such as accuracy become misleading & cannot be used to judge model performance. A banking environment prioritises financial risk reduction, customer trust, & operational efficiency. These metrics form the basis for evaluating fraud detection models from a financial institute’s perspective.

5.1.1 Accuracy

Accuracy measures the proportion of all correct predictions. *It is almost meaningless in fraud detection. A model that labels every transaction as “not fraud” would achieve more than 99% accuracy while detecting no fraud at all. Therefore, accuracy does not capture the risk of missed fraud & must not be used as the primary metric.*

5.1.2 Precision

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}.$$

It indicates how often the model’s fraud alerts are actually correct. *A low-precision model generates many false alarms, causing unnecessary card blocks, customer dissatisfaction, & increased workload for fraud analysts.* From a banking viewpoint, precision reflects **operational efficiency** & the impact of fraud alerts on genuine customers.

5.1.3 Recall

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}.$$

Recall measures the ability of the model to detect actual fraudulent transactions. *A missed fraud (false negative) directly translates to financial losses for the institution.* **For this reason, banks typically prioritise recall over precision. High recall ensures the system catches as many fraudulent activities as possible, reducing monetary risk.**

5.1.4 F1 Score

$$F1 = 2 \times \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

It provides a balanced single-number summary of the trade-off between correctly identifying fraud (recall) & avoiding false alarms (precision).

5.1.5 Receiver Operating Characteristic (ROC) AUC

The ROC-AUC score evaluates how well the model separates fraudulent & non-fraudulent transactions across all possible thresholds. It is useful for comparing the general discriminatory power of different models. *However, ROC-AUC can be misleading in imbalanced datasets, because it is strongly influenced by the large number of true negatives, which are relatively unimportant in fraud analysis.*

5.1.6 Precision-Recall AUC (PR-AUC)

PR-AUC is the most informative metric for imbalanced fraud datasets. It focuses only on positive-class performance (fraud detection) by summarising the relationship between precision & recall across thresholds. A higher PR-AUC indicates a model that detects fraud while minimizing false alarm rates.

5.1.7 Confusion Matrix

The confusion matrix provides an interpretable breakdown of:

Confusion Matrix		
	Predicted 0	Predicted 1
True 0	True Negative (correctly detected legit)	False Negative (missed Frauds, leading to losses)
True 1	False Positive (legitimate customers tagged as fraud)	True Positive (correctly detected frauds)

From a managerial perspective:

- *False Negatives* are the highest financial risk.
- *False Positives* are the highest customer experience & operational risk.

5.1.8 Most Important Metrics for Model Selection

The order of importance:

1. **PR-AUC** — the most reliable metric for imbalanced fraud datasets.

2. **Recall** — to minimise financial losses due to missed fraud.
3. **F1 Score** — secondary combined metric.
4. **ROC-AUC** — useful but less informative under severe imbalance.
5. **Precision** — to reduce customer inconvenience & operational costs.
6. **Accuracy** — not meaningful for fraud detection.

5.2 Model Experimentation

We trained the following models:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost
- LightGBM
- SVM

5.2.1 Logistic Regression

The objective of this analysis is to establish a linear baseline model using logistic regression.

Mathematical Framework

The Logistic Regression model models the probability of fraud $P(y = 1|x)$ using the Sigmoid function $\sigma(z)$.

Linear Combination (z)

For a feature vector x (containing Amount, Hour, Category, etc.), the model learns a weight vector w & a bias term b . The magnitude & direction of these weights indicate how strongly a feature influences the fraud prediction. The linear score z is calculated as:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = w^T x + b \quad (1)$$

Table 1: Logistic Regression Feature Weights

Feature d	Weight (w)
city_pop	7.63×10^{-8}
zip	-6.46×10^{-9}
amt	4.80×10^{-9}
unix_time	-3.84×10^{-9}
hour	1.17×10^{-11}
category	1.08×10^{-11}
day	3.57×10^{-12}
merch_long	3.25×10^{-12}
long	3.20×10^{-12}
month	-2.85×10^{-12}
lat	1.22×10^{-12}
merch_lat	1.12×10^{-12}
gender	5.46×10^{-13}

The Sigmoid Activation The linear score z implies a range of $(-\infty, \infty)$. To convert this into a probability range $[0, 1]$, we apply the *Sigmoid function*:

$$y = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

If $\sigma(z) \geq 0.5$, the model predicts Fraud; otherwise, it predicts Not Fraud.

Table 2: Default vs. Tuned Hyperparameters

Configuration	Accuracy	Precision	Recall	F1 Score	ROC-AUC	PR-AUC
Default	0.9961	0.0000	0.0000	0.0000	0.5497	0.0041
Tuned	0.9530	0.0593	0.7506	0.1099	0.8270	0.1364

Confusion Matrix for Logistic Regression

Default Logistic Regression		
	Predicted 0	Predicted 1
True 0	553574	0
True 1	2145	0

Tuned Logistic Regression		
	Predicted 0	Predicted 1
True 0	528017	25557
True 1	535	1610

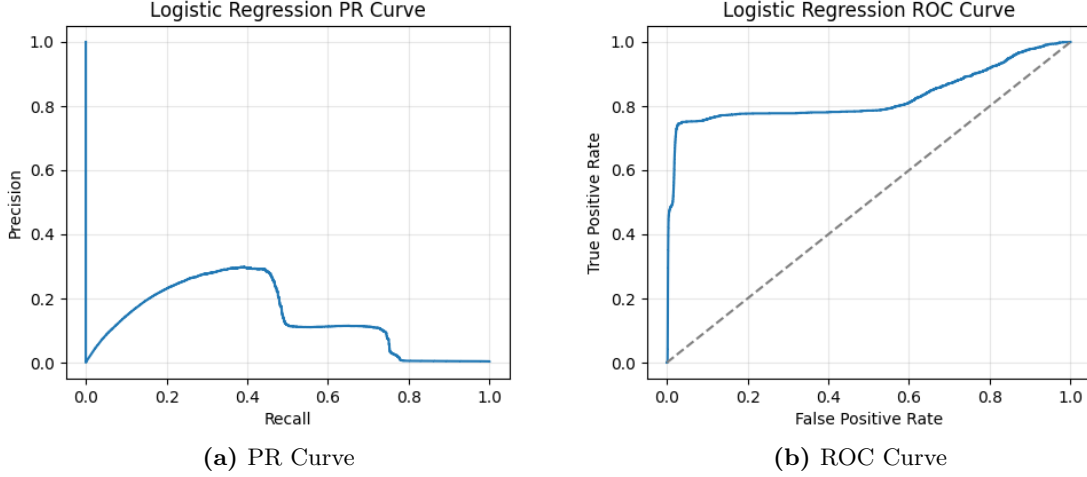


Figure 5: Logistic Regression (Tuned)

5.2.2 Decision Tree Classifier

For this analysis, we implemented two distinct variations of the Decision Tree model to compare their efficacy: the **Gini index** & the **Entropy (Information Gain)**.

Mathematical Splitting Criteria The core logic of the decision tree involves recursively partitioning the feature space. The quality of a split is determined by the impurity of the resulting nodes.

1. **Gini Impurity** measures the likelihood of an incorrect classification of a new instance of a random variable if that new instance were randomly classified according to the distribution of classes.

$$Gini(D) = 1 - \sum_{i=1}^C (p_i)^2 \quad (3)$$

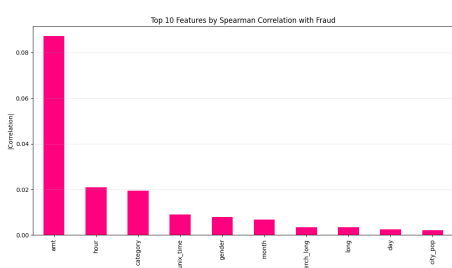
Where p_i is the probability of an item being classified to a particular class i . Gini is computationally faster as it avoids logarithmic calculations.

2. **Entropy & Information Gain:** Entropy measures the amount of randomness or uncertainty in the data. The algorithm selects the split that maximises the *information gain*.

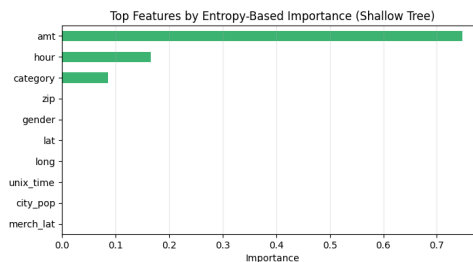
$$Entropy(D) = - \sum_{i=1}^C p_i \log_2(p_i) \quad (4)$$

While Entropy tends to produce more balanced trees, it is computationally more intensive.

Justification for Feature Selection in the Decision Tree Model The decision tree was intentionally trained on a reduced subset of features, **amt**, **hour**, & **category** to balance predictive performance, interpretability, & generalisation.



(a) Top 10 Features (by correlation)



(b) Top Features by Entropy-based Importance (Shallow Tree)

Figure 6: Feature Importances

- Prior correlation analysis & feature-importance rankings consistently identified transaction amount as the strongest individual predictor.
- Temporal behaviour signals fraudulent transactions that tend to cluster during atypical times when legitimate card usage is low.
- The **category** feature captures merchant-level transaction information, enabling the model to distinguish between inherently low-risk & high-risk spending areas.

Restricting the tree to these high-signal features **reduces noise from weakly informative variables, mitigates overfitting** & results in a more stable decision boundary. From an operational standpoint, a compact feature set also enhances model transparency & auditability.

Hyperparameter Tuning A critical challenge in decision tree modelling is the *bias-variance tradeoff*. A shallow tree may fail to capture the underlying patterns (high bias), while a deep tree may memorise the training noise (high variance). To determine the optimal complexity, we conducted a *Validation Curve analysis*, iterating the **max_depth** hyperparameter from 1 to 20.

Figure 7 illustrates the relationship between tree depth & model performance (F1-score).

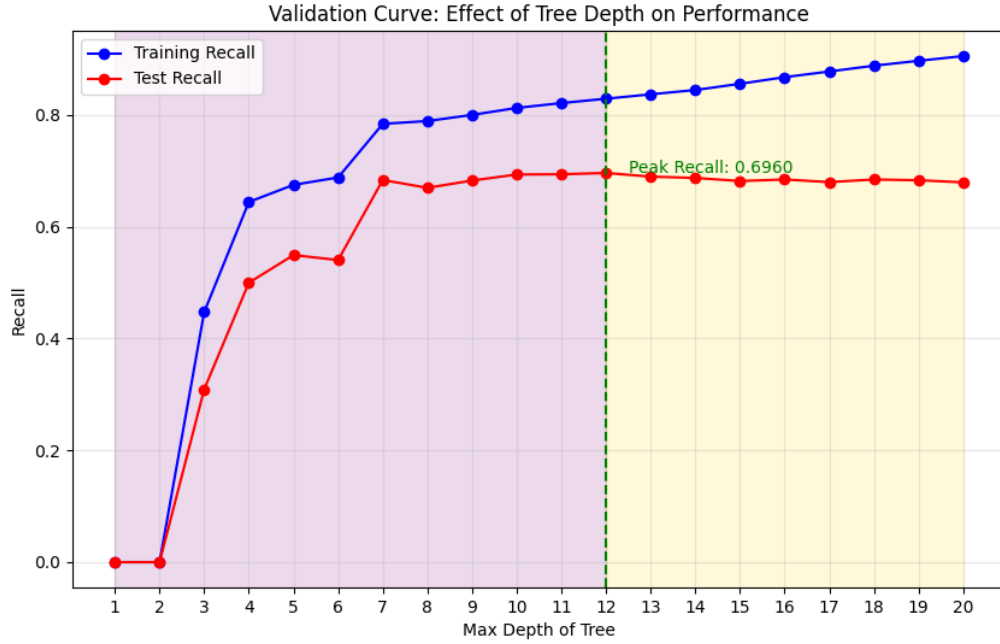


Figure 7: Validation Curve for Decision Tree Depth. The graph visually segments the model complexity into three distinct regions: Underfitting, Optimal Fit, & Overfitting.

The analysis of the Validation Curve reveals three distinct behavioural regions:

1. **Region of Underfitting:** At low depths ($1 \leq d < d_{opt}$), both the **Training Score** & **Testing Score** are suboptimal. In this region, the model is too simple to capture the complex non-linear relationships inherent in fraud patterns. The high bias results in significant misclassification of both fraud & non-fraud cases.
2. **The Optimal Depth (d_{opt}):** The optimal depth is at the point where the Test F1-Score is maximised. At this specific depth, the model achieves the best balance, capturing the signal without fitting to the noise. The optimal depth is **12**.
3. **Region of Overfitting:** Beyond the optimal depth ($d > d_{opt}$), the **training score** continues to approach 1 (perfect memorisation), while the **test score** plateaus or degrades. In this region, the model essentially "memorises" the training data, including outliers & noise. This results in poor generalisation to unseen data.

Table 4: Default vs. Tuned Hyperparameters

Configuration	Accuracy	Precision	Recall	F1 Score	ROC-AUC	PR-AUC
Default	0.9983	0.8316	0.6951	0.7572	0.9884	0.7914
Tuned	0.9981	0.7909	0.6895	0.7367	0.9481	0.7553

Based on this analysis, we finalised the model **hyperparameters** at `max_depth=12`, providing the highest generalisation performance on the validation set.

Looking at the performance comparison table for the two variants of the decision tree, we can clearly see that the entropy model performs better in terms of F1 score, ROC-AUC &

PR-AUC. The Gini model is better when comparing recall, but only by 0.0033, which is miniscule. Therefore, we use entropy as the criterion for the decision tree.

Table 5: Performance Comparison: Decision Tree Gini vs. Entropy

Decision Tree	Accuracy	Precision	Recall	F1 Score	ROC-AUC	PR-AUC
Gini Index	0.9982	0.8199	0.6984	0.7543	0.9860	0.7907
Entropy	0.9983	0.8316	0.6951	0.7572	0.9884	0.7914

Decision Tree (Entropy) Confusion Matrix		
	Predicted 0	Predicted 1
True 0	553272	302
True 1	654	149

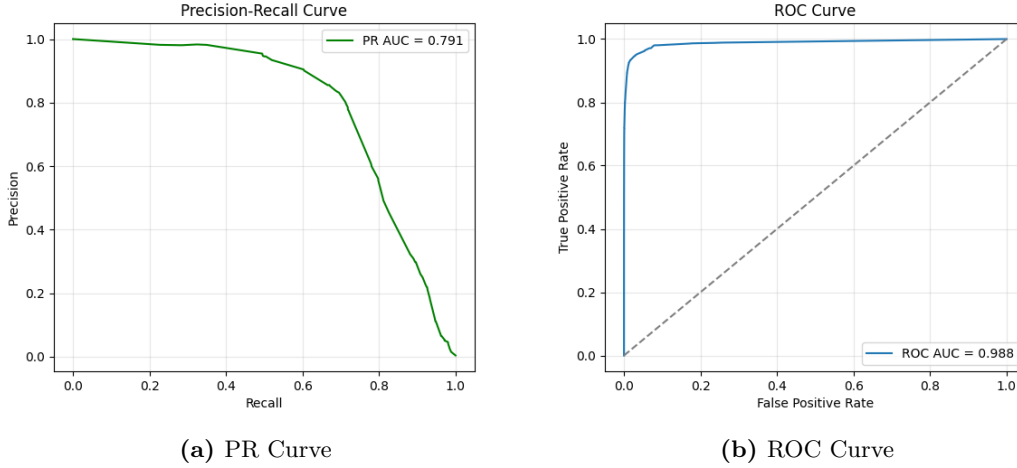


Figure 8: Entropy Decision Tree

The **ROC-AUC** indicates that the classifier has a very strong ability to rank suspicious transactions higher than legitimate ones.

The **PR-AUC & F1 score** further confirm strong performance under class imbalance & a good balance between precision & recall. Since fraud cases are rare, PR-AUC becomes more informative than ROC-AUC. A score close to 0.80 shows that the model maintains high precision while still capturing a meaningful portion of true frauds. This suggests the model remains reliable even when the number of non-fraud cases overwhelmingly dominates. This balance is essential as the bank avoids excessive false alarms while still identifying a substantial portion of fraudulent transactions.

5.2.3 Extreme Gradient Boosting (XGBoost)

To further improve predictive performance & reduce variance, we implemented the XGBoost algorithm. Unlike the decision tree, which builds a single complex estimator, XGBoost is an ensemble method that constructs a series of weak learners (shallow trees) sequentially.

Theoretical Framework By fitting each new tree to the residual errors of the previous ensemble, XGBoost effectively "corrects" the mistakes of prior iterations, leading to a highly robust decision boundary.

Table 6: Performance Metrics: XGBoost Classifier

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC	PR-AUC
Default	0.9981	0.8827	0.5930	0.7094	0.9938	0.7907
Tuned	0.9980	0.8308	0.5930	0.6921	0.9913	0.7578

Performance Evaluation

Analysis of Results

1. **Precision :** A precision of 88% indicates that when XGBoost flags a transaction as fraudulent, it is highly likely to be correct. In an operational context, this is invaluable, as it drastically reduces the "false alarm" rate, minimising customer annoyance & administrative costs.
2. **Recall Trade-off (0.5930):** While the model is highly precise, the recall is lower (59.3%) compared to the decision tree. It prefers to miss some subtle fraud cases (False Negatives) rather than incorrectly flag legitimate customers.
3. **ROC-AUC (0.9938):** The near-perfect ROC score suggests that the model is excellent at ranking probabilities. The discrepancy between the high ROC & the moderate recall suggests that while the model separates the classes well, the default decision threshold ($p > 0.5$) might be too high. Adjusting this threshold could recover more recall at the cost of some precision.

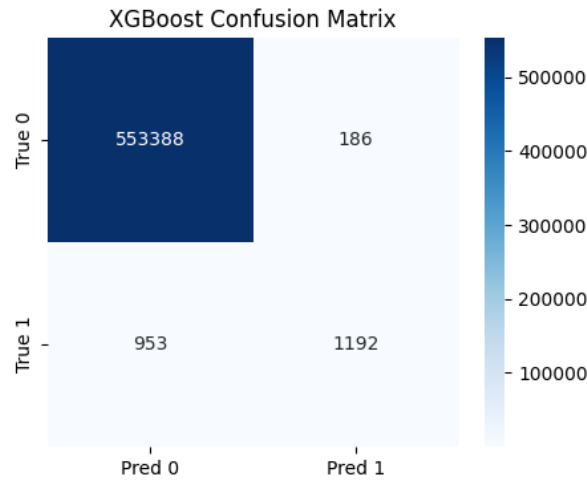


Figure 9: Confusion Matrix (XGBoost). The high number of true negatives & low false positives confirms the model's high specificity & precision.

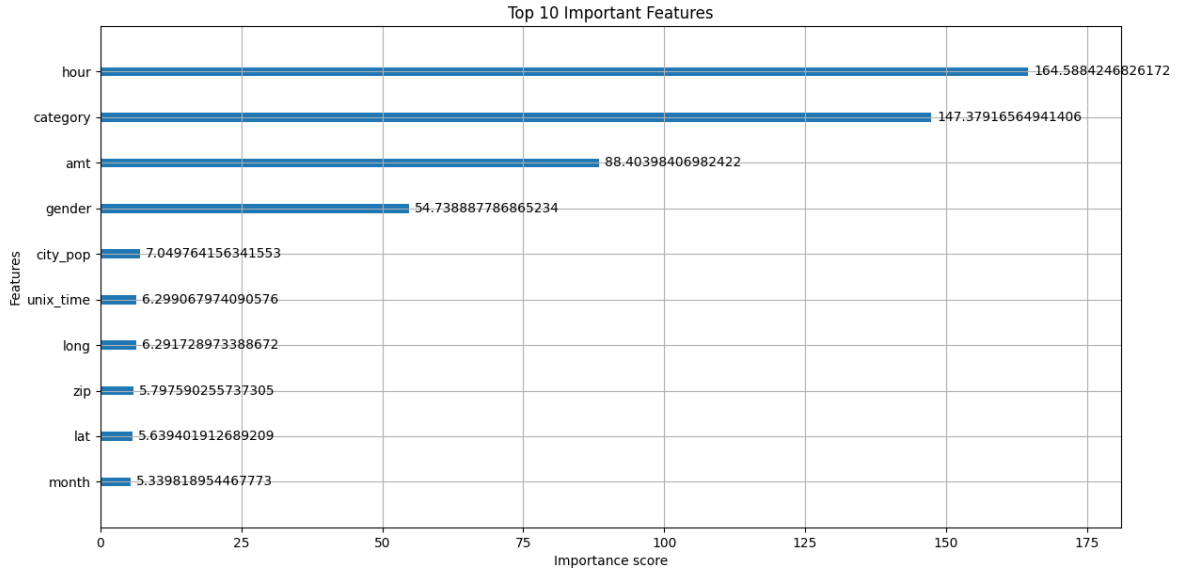


Figure 10: Feature Importance (XGBoost). Unlike linear models, XGBoost captures non-linear interactions between features such as 'Amount' & 'Time of Day'.

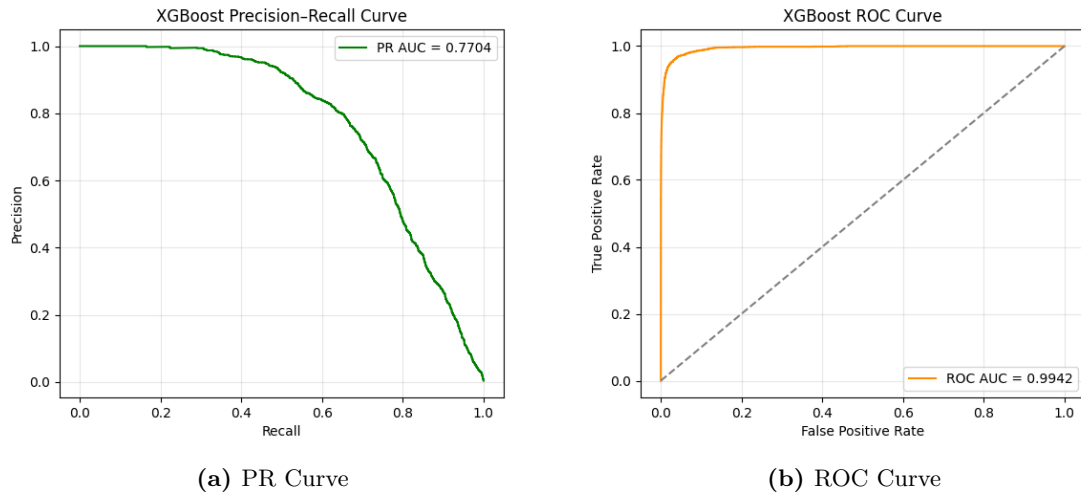


Figure 11: Entropy Decision Tree

In conclusion, XGBoost serves as our **"high-confidence"** detector. While it catches fewer total fraud cases than the decision tree, the alerts it generates are significantly more trustworthy, making it an ideal candidate for automated blocking systems where false positives are unacceptable.

5.2.4 Random Forest Classifier

While the single decision tree provided a strong baseline, it is inherently prone to overfitting (high variance). To solve this, we implemented a *Random Forest Classifier*, an *ensemble learning method* that operates by constructing a multitude of decision trees during training & outputting the class that is the mode of the classes of the individual trees.

Theoretical Framework: Bagging Random Forest relies on **Bagging** (*Bootstrap Aggregating*). It introduces randomness in two ways to ensure trees are de-correlated:

1. **Data Sampling:** Each tree is trained on a random subset of the data sampled with replacement.
2. **Feature Sampling:** At each split, the algorithm considers only a random subset of features rather than all available features.

Performance Evaluation The Random Forest model was trained with max 500 estimators & a maximum depth of 16. The performance metrics are detailed below:

Table 7: Default v/s Tuned

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC	PR-AUC
Default	0.9913	0.2919	0.8765	0.4379	0.9913	0.7374
Tuned	0.9970	0.9813	0.2200	0.3595	0.9619	0.7429

Precision-Recall Balance: For the default model, it has strong performance in imbalanced settings. Shows that the model has good fraud-detection potential if thresholds are tuned. For the tuned model, slightly higher than the default model. Indicates that while raw recall is low at threshold 0.5, the model's scoring distribution still contains useful signal; recall could be improved by lowering the decision threshold.

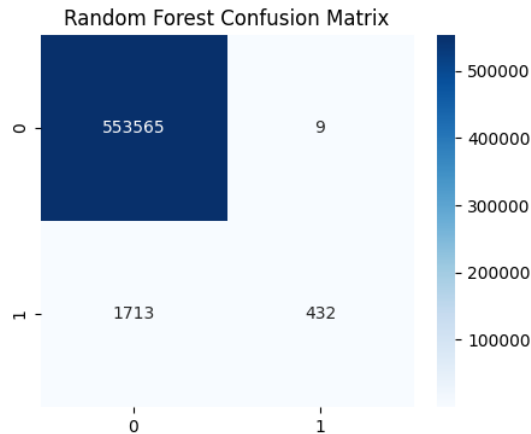


Figure 12: Random Forest Confusion Matrix

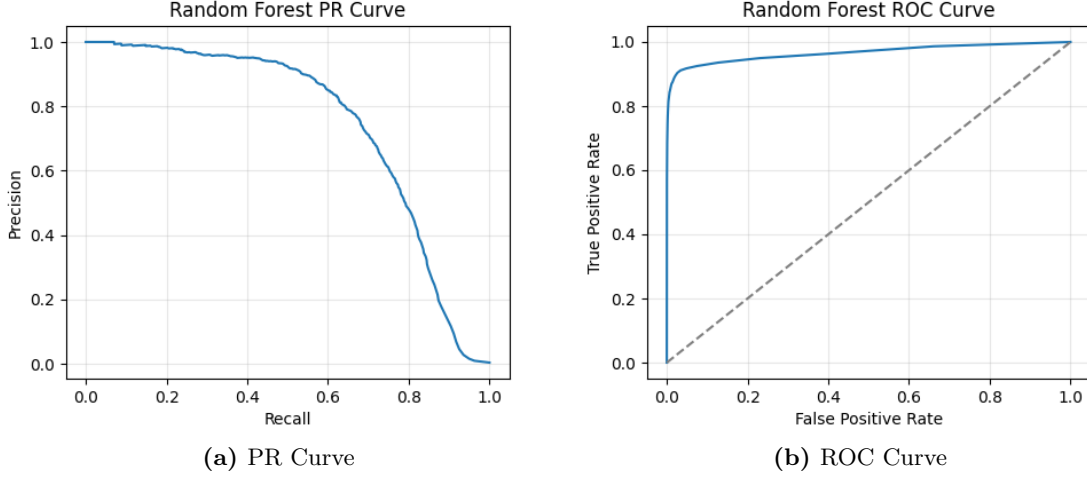


Figure 13: Random Forest Classifier

A bank would never operate at either extreme because both ends of the precision–recall spectrum create serious financial & operational risks. A model with extremely high recall but low precision overwhelms the fraud department with false alarms, blocks legitimate customer transactions, increases support-center load, & damages customer trust—ultimately costing the bank more than the fraud it’s trying to prevent.

On the other hand, a model with extremely high precision but low recall misses most fraudulent transactions entirely, directly exposing the bank to financial losses & bad reputation.

5.2.5 Support Vector Machine (SVM)

Support Vector Machines (SVM) is an error-based learning model that aims to identify a decision boundary which separates the two target classes in the dataset. SVMs explicitly search for the separating hyperplane that maximises the margin, which is the perpendicular distance between the decision boundary & the nearest training instances. These instances lying along the margin extents are known as the support vectors, & they uniquely determine the model’s decision surface.

Maximum-Margin Principle A separating hyperplane is expressed as:

$$w_0 - w^T d = 0$$

where d denotes the descriptive feature vector. For binary classification, the target labels are encoded as $t_i \in \{-1, +1\}$. The margin constraints for all training instances become:

$$t_i (w_0 - w^T d_i) \geq 1$$

The distance from a support vector to the decision boundary is:

$$\text{dist}(d_i) = \frac{1}{\|w\|}$$

Since the total margin size is $\frac{2}{\|w\|}$, training an SVM corresponds to the quadratic optimization problem:

$$\min_w \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad t_i (w_0 - w^T d_i) \geq 1 \quad \forall i$$

Maximizing the margin improves generalization, which is essential in fraud detection where classes are highly imbalanced & decision boundaries must be robust to noise.

Kernel Trick for Nonlinear Fraud Patterns Fraud data is rarely linearly separable. To allow nonlinear decision boundaries, SVMs apply a basis transformation $\phi(d)$, but instead of computing dot products in this higher-dimensional space, the *kernel trick* is used:

$$K(d_i, q) = \phi(d_i)^T \phi(q)$$

In this project, we selected the Gaussian Radial Basis Function (RBF) kernel:

$$K(d_i, q) = \exp(-\gamma \|d_i - q\|^2)$$

The RBF kernel captures complex, localised fraud behaviour patterns such as unusual combinations of transaction amount, time-of-day, & geolocation.

SVM Model Formulation The SVM prediction function is given by:

$$M(q) = \sum_{i=1}^s (t_i \alpha_i K(d_i, q)) - w_0$$

where α_i are Lagrange multipliers learned during training & s is the number of support vectors. Importantly, only the support vectors influence predictions, which reflects the geometric nature of the algorithm.

Soft Margin & Practical Training Real-world fraud data contains overlap between classes, so a *soft margin* SVM is used. This introduces a regularization parameter C which balances margin width against classification errors. Larger C penalizes misclassification more heavily, producing a tighter decision boundary; smaller C yields a wider, more generalizable margin.

In our implementation, we tuned the hyperparameters:

$$C \in \{0.1, 1, 10, 50\}, \quad \gamma \in \{\text{scale}, 0.01, 0.001\}$$

using `RandomizedSearchCV`, with the RBF kernel.

Why We Chose SVM We selected SVM for this project because:

- It is highly effective in high-dimensional spaces, thus relevant for transaction data.
- The maximum-margin principle makes the classifier robust to noise.
- The RBF kernel captures nonlinear fraud patterns that simpler models may miss.
- SVMs are less prone to overfitting compared to purely tree-based models.

Performance Evaluation To assess the effectiveness of the SVM model, we analysed three key performance visualisations: the confusion matrix, the ROC curve, & the Precision–Recall (PR) curve.

Confusion Matrix Figure 14 presents the confusion matrix for the SVM model trained without hyperparameter tuning. The classifier correctly identifies the vast majority of non-fraudulent transactions ($TN = 553,315$), demonstrating strong performance on the dominant class. It detects 618 fraudulent transactions (TP), but still misses 1,527 fraud cases (FN), which is expected given the class imbalance and the absence of tuned parameters. The number of false positives (259) remains relatively low, indicating that the model is conservative when predicting fraud. Overall, the model captures meaningful fraud patterns, but its recall is limited by not having optimized C and γ .

Overall, the matrix reflects a conservative classifier that prioritises minimising false alarms while still retrieving a reasonable fraction of actual frauds.

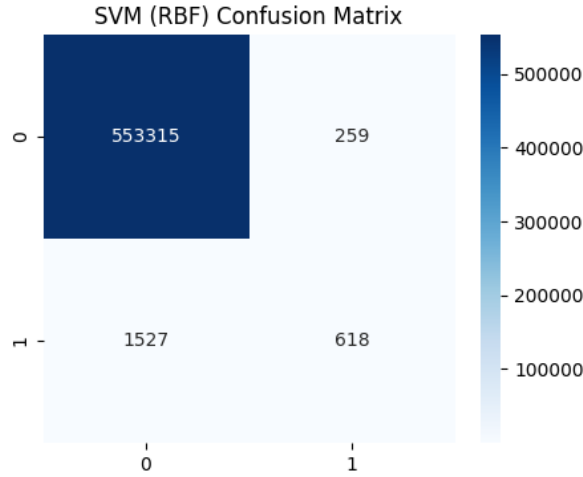


Figure 14: SVM Confusion Matrix

ROC Curve The ROC curve in Figure 15 shows a smooth and consistently rising trajectory, with the classifier achieving strong true positive rates across most false positive thresholds. Although the curve does not reach the performance levels observed in tuned tree-based models, it still demonstrates solid ranking ability. This confirms that the SVM decision boundary generalizes reasonably well to unseen data despite using default hyperparameters.

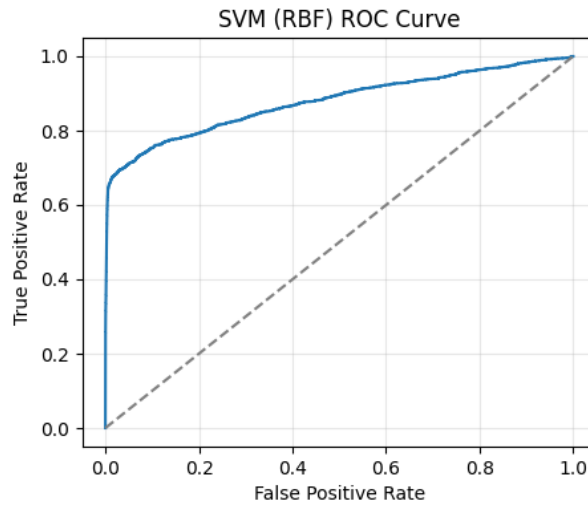


Figure 15: ROC Curve for SVM Classifier

Precision-Recall Curve The Precision-Recall curve in Figure 16 provides deeper insight into performance under class imbalance. Precision is high at very low recall levels, indicating that the model is confident about the small number of fraud cases it does capture. However, precision declines as recall increases, reflecting the difficulty of identifying more fraud cases without introducing additional false positives. This behaviour is characteristic of untuned SVMs on imbalanced datasets and highlights the trade-off between sensitivity and reliability.

Summary Overall, the SVM model demonstrates reasonable discrimination ability even without hyperparameter tuning. Its ROC curve indicates solid ranking performance, showing that the model can separate fraudulent and non-fraudulent transactions better than random

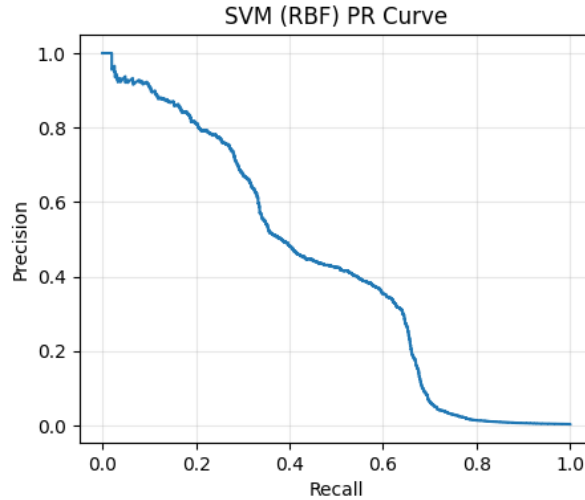


Figure 16: Precision–Recall Curve for SVM Classifier

guessing. The confusion matrix reinforces the challenge of fraud detection: while the classifier is highly accurate for the non-fraud class, it still misses a substantial number of fraud cases due to the extreme imbalance in the dataset and the use of default model settings. Nevertheless, the SVM captures meaningful fraud patterns and provides a useful baseline for comparison against more complex or tuned models.

5.2.6 LightGBM

LightGBM (Light Gradient Boosting Machine) is a gradient-boosted decision tree model designed for high efficiency on large datasets. It builds an ensemble of shallow decision trees, where each tree attempts to correct the errors of the previous ones. Unlike traditional boosting methods, LightGBM grows trees *leaf-wise* rather than level-wise. This allows the model to focus on the most informative splits, making it well-suited for complex, imbalanced datasets such as fraud detection.

Why We Chose LightGBM We selected LightGBM because:

- It trains significantly faster than XGBoost & Random Forest on large datasets.
- The leaf-wise growth strategy captures subtle fraud patterns that may be missed by simpler models.
- It handles class imbalance effectively using the `class_weight=balanced` option.
- It provides feature importance scores, aiding interpretability.

Hyperparameter Tuning To improve generalization, we tuned LightGBM using `RandomizedSearchCV`, focusing on parameters that control model complexity: number of trees, learning rate, number of leaves, and sampling ratios. Since boosting models can easily overfit, tuning was important to balance model depth with regularization. The search was performed on a reduced subset of the training data to reduce computational cost while still providing reliable estimates of good parameter values. The tuned model was later trained on the full dataset.

LightGBM Evaluation LightGBM (Light Gradient Boosting Machine) is a highly efficient gradient-boosted decision tree algorithm designed for large-scale datasets. Unlike traditional level-

wise tree growth, LightGBM grows trees leaf-wise, enabling it to focus on the most informative splits while achieving strong predictive performance. This property makes it particularly effective for fraud detection, where subtle nonlinear patterns must be captured from highly imbalanced data.

We selected LightGBM for this project because:

- It trains significantly faster than XGBoost & Random Forest on large datasets.
- The leaf-wise strategy captures complex fraud behaviour which simpler models may overlook.
- It handles class imbalance effectively using `class_weight=balanced`.
- It provides interpretable feature importances.

Hyperparameter tuning was performed using `RandomizedSearchCV`, focusing on parameters such as number of trees, learning rate, number of leaves, & subsampling ratios. A reduced subset of the training data was used for tuning to reduce computational cost while preserving representativeness.

Confusion Matrix Figure 17 shows the confusion matrix for the tuned LightGBM model. The classifier achieves a very low number of false negatives (FN = 558), meaning that it successfully captures a large proportion of fraudulent transactions. This is crucial in fraud detection, where missing a fraud case is far more costly than raising a false alarm. At the same time, the number of false positives (FP = 1036) remains manageable relative to the size of the dataset.

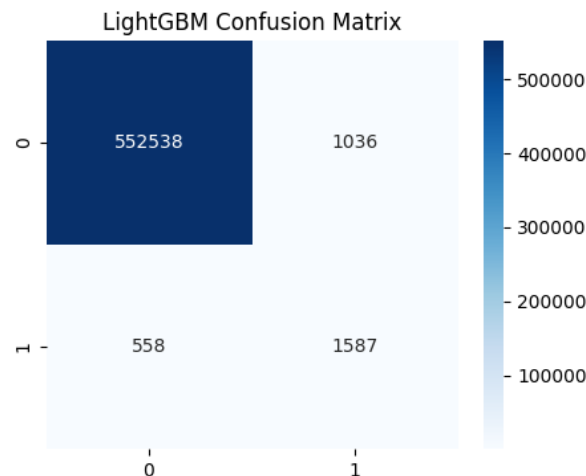


Figure 17: LightGBM Confusion Matrix

ROC Curve Figure 18 presents the ROC curve for LightGBM. The curve stays well above the diagonal baseline, reflecting a high true positive rate even at low false positive rates. This indicates that the model assigns consistently higher scores to fraudulent transactions than to legitimate ones. The strong AUC value aligns with LightGBM's ability to separate the two classes effectively.

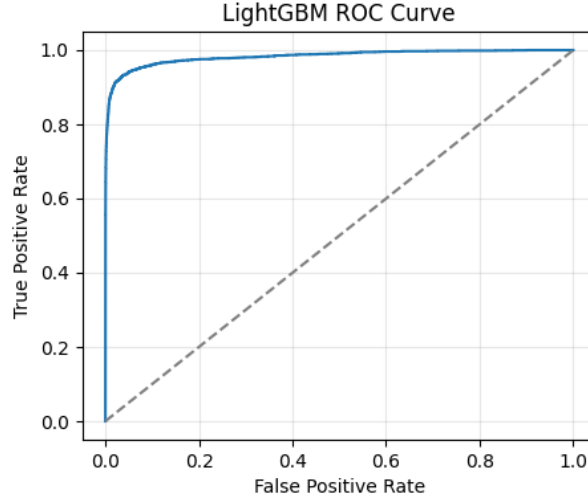


Figure 18: LightGBM ROC Curve

Precision–Recall Curve The Precision–Recall curve, shown in Figure 19, provides a more informative view for imbalanced datasets. LightGBM maintains high precision at moderate recall levels, showing that when the model predicts fraud, it is usually correct. Although precision declines as recall increases, the curve significantly outperforms random classification, demonstrating the model’s effectiveness in prioritizing likely fraud cases.

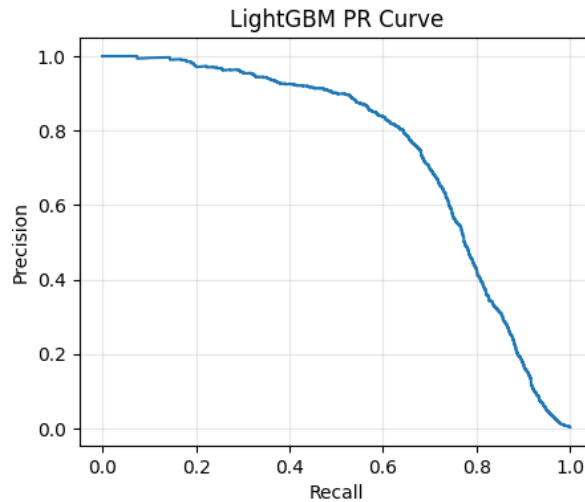


Figure 19: LightGBM Precision–Recall Curve

5.2.7 SHAP & LIME

In this project, we used SHAP & LIME on the XGBoost model because of two main reasons. Firstly, XGBoost is one of the strongest-performing models in our pipeline, & therefore it is the most meaningful choice for detailed understanding analysis. Since fraud detection models are often used in high-stakes settings, understanding why the best model makes certain predictions is essential.

Second, SHAP provides exact & efficient explanations specifically for tree-based models through its **TreeExplainer**. This makes SHAP computationally feasible for XGBoost but significantly more expensive for models such as SVM, Logistic Regression, or LightGBM. Similarly, LIME

was applied to XGBoost because it offers local explanations for individual predictions & is model-agnostic, allowing us to illustrate how a specific fraud decision is made without need to explain all models in the pipeline.

Overall, XGBoost was selected for interpretability because it achieved high performance and is well-supported by SHAP & LIME, making it the most practical & informative model for explanation in our project.

5.3 Summary of Results

Table 8: Comprehensive Model Comparison: Default vs. Tuned Hyperparameters

Algorithm	Configuration	Accuracy	Precision	Recall	F1 Score	ROC-AUC	PR-AUC
Logistic Regression	Default	0.9961	0.0000	0.0000	0.0000	0.5497	0.0041
	Tuned	0.9530	0.0593	0.7506	0.1099	0.8270	0.1364
Decision Tree	Default	0.9983	0.8316	0.6951	0.7572	0.9884	0.7914
	Tuned	0.9981	0.7909	0.6895	0.7367	0.9481	0.7553
XGBoost	Default	0.9981	0.8827	0.5930	0.7094	0.9938	0.7907
	Tuned	0.9980	0.8308	0.5930	0.6921	0.9913	0.7578
Random Forest	Default	0.9913	0.2919	0.8765	0.4379	0.9913	0.7374
	Tuned	0.9970	0.9813	0.2200	0.3595	0.9619	0.7429
SVM	Default	0.9968	0.7855	0.2219	0.3461	0.8501	0.3773
	Tuned	0.9964	0.5717	0.3142	0.4055	0.8500	0.3719

6 Discussion and Analysis

This section synthesises the experimental results across all models & frames them within the practical constraints of real-world credit card fraud detection. Rather than focusing on isolated metric maximisation, we interpret model behaviour through the lenses of class imbalance, operational risk in real life, and decision-making under uncertainty.

6.1 Why SMOTE Was Ultimately Removed

Although Synthetic Minority Over-sampling Technique (SMOTE) is a widely used technique for handling class imbalance, our experiments demonstrated that its application in this dataset introduced more harm than benefit. Initial interim results showed that while SMOTE significantly increased recall, it simultaneously caused a severe degradation in precision and PR-AUC.

- **Distortion of feature correlations:** As observed in the EDA, SMOTE artificially inflated the correlation between features (e.g., `amt`) and the target variable. This indicates a critical diagnostic that observed distortion in feature–target relationships. This indicates that synthetic samples introduced by SMOTE did not preserve the original data manifold, **effectively creating fraud patterns that do not exist in real transactional behaviour.**
- **Precision Degradation:** Models trained on SMOTE-augmented data exhibited a drastic drop in precision. In a banking context, a model that "hallucinates" fraud patterns results in an excessive False Positive Rate (FPR) and reduces customer trust. The cost of investigating these false alarms outweighs the benefit of marginally higher recall.

Consequently, SMOTE was deliberately excluded from the final pipeline in favour of cost-aware evaluation and threshold tuning on the original data distribution.

6.2 Hyperparameter Tuning using RandomizedSearchCV

To optimize model performance while maintaining computational feasibility, hyperparameter tuning was performed using `RandomizedSearchCV` rather than exhaustive grid search. This choice was particularly important given the scale of the dataset and the high-dimensional hyperparameter spaces of ensemble models such as Random Forest and XGBoost.

Why RandomizedSearchCV? Unlike `GridSearchCV`, which evaluates all possible combinations of hyperparameters, `RandomizedSearchCV` samples a fixed number of hyperparameter configurations from predefined distributions. This approach offers two key advantages:

- **Computational Efficiency:** Randomised search explores a much wider region of the hyperparameter space with significantly fewer evaluations, making it suitable for large datasets and complex models.
- **Better Coverage of Important Regions:** Random sampling increases the probability of discovering high-performing regions without exhaustively searching uninformative combinations.

Why Do Metrics Change Across Runs? An important observation is that final evaluation metrics may vary slightly each time `RandomizedSearchCV` is executed. Each run samples a different subset of hyperparameter combinations from the defined distributions. As a result, the optimal configuration identified in one run may differ slightly from another.

6.3 Hyperparameter Tuning: What Was Kept and What Was Rejected

Hyperparameter tuning was applied selectively, guided by two key metrics: **PR-AUC** and **Recall**. While accuracy and ROC-AUC were monitored, they were not used as primary selection criteria due to the extreme class imbalance.

- **Decision Tree:** For this model, tuning the maximum depth proved essential to control overfitting. Validation curves clearly showed a *bias-variance transition*, allowing us to identify an optimal depth that maximised generalisation performance rather than training memorisation.
- **Random Forest (The Precision Shift):** The default Random Forest acted as a high-recall engine (Recall $\approx 87\%$, Precision $\approx 29\%$). However, upon tuning, the model behaviour inverted (Recall $\approx 22\%$, Precision $\approx 98\%$). This indicates that the tuning process, driven by the F1-score and precision-recall objectives, prioritised certainty over coverage. While the tuned model is extremely trustworthy, the drop in recall suggests that ensemble trees require careful threshold calibration rather than just hyperparameter optimisation.
- **Logistic Regression:** The default linear model failed completely (Recall 0.0) due to the overwhelming majority class. Tuning was not optional but mandatory; adjusting the decision threshold was important to make the model even functional.
- **XGBoost & LightGBM:** These gradient-boosting models showed the most robustness. Tuning refined their performance but did not drastically alter their behaviour compared to Random Forest. This suggests that boosting algorithms are inherently better suited for learning complex, non-linear fraud patterns in imbalanced feature spaces.

This highlights a key methodological point: hyperparameter tuning is not about maximising metrics indiscriminately, but about selecting operating points aligned with business risk. Models were retained only if tuning improved PR-AUC or offered a meaningful trade-off between false positives and false negatives.

6.4 Error Analysis: False Positives & False Negatives

An analysis of misclassified instances reveals clear patterns rooted in the dataset properties:

- **False Negatives (Missed Fraud):**

- *Low-Value Transactions:* The models frequently missed fraud transactions with very low amounts. Fraudsters often test cards with small purchases before making large ones. These "testing" transactions mimic normal spending behaviour, lacking the distinct signal of high-value theft.
- *Mimicry:* Fraud occurring at "normal" hours (e.g., midday) was harder to detect than late-night fraud, as it overlaps with the density of legitimate transactions.

- **False Positives (False Alarms):**

- *Outliers in Legit Behaviour:* Legitimate customers making purchases in rare categories or unusual locations were often flagged. This highlights the limitation of relying heavily on geospatial distance and category encoding without past customer profiles.
- *High-Value Legit Transactions:* The models learnt a strong correlation between **amt** and fraud. Consequently, legitimate high-value purchases (e.g., luxury goods, travel) triggered false alarms.

This asymmetry solidifies the importance of PR-AUC and confusion matrix analysis over accuracy. A single false negative can result in direct financial loss, while excessive false positives degrade customer experience and increase operational cost.

6.5 Why Banks Never Operate at Either Extreme

A high-recall, low-precision system, the default Random Forest. The indirect costs of customer churn and reputational damage may exceed the fraud losses avoided. Conversely, a high-precision, low-recall system, tuned Random Forest, misses the majority of fraud cases, exposing the bank to direct financial losses. The optimal solution lies in selecting a balanced operating region, often achieved through threshold tuning rather than model replacement.

6.6 Model-Specific Behavioral Insights

- Across experiments, tree-based ensemble models (**XGBoost** and **LightGBM**) consistently achieved the strongest PR-AUC and ROC-AUC scores. This indicates superior ranking ability under class imbalance, with their ability to model nonlinearity between transaction amount, timing and category.
- **Logistic regression** served as a valuable baseline but failed to capture nonlinear fraud patterns. **Decision trees** offered strong interpretability and competitive performance but required careful pruning to avoid overfitting.
- **SVMs** demonstrated robust ranking behaviour but were computationally constrained and sensitive to kernel and sampling choices.

6.7 Limitations and Risks

Despite strong model performance, several limitations inherent to the dataset & modelling pipeline that should be acknowledged.

1. **Extreme Class Imbalance:** With fraud representing less than 0.3% of the data, the model's performance is highly sensitive to thresholding. The scarcity of positive samples limits the model's ability to learn rare or novel attack vectors.

2. **Data Age:** The dataset spans 2019–2020. Real-world fraud is adversarial and evolves rapidly. Models trained on pre-2021 data may fail to detect modern fraud tactics (e.g., bot attacks, synthetic identity fraud, AI frauds) or adapt to post-COVID consumer taste & behaviour changes.
3. **Absence of Personally Identifiable Information & History:** To preserve privacy, columns like names and specific merchant IDs were removed. However, real-world banking systems rely heavily on *historical profiles* (e.g., "Has this user shopped here before?"). The absence of these features limits the model to examining transactions in isolation rather than as part of a sequence.
4. **Static Threshold:** all predictions carry business risk. Static thresholds are therefore insufficient. Adaptive, segment-specific thresholds are more appropriate in production systems.

6.8 Future Improvements and Recommendations

To transition this system from a prototype to a production-level banking solution, we thought of the following improvements:

- **Advanced Feature Engineering:** Moving beyond static features, we should engineer *velocity features* (e.g., number of transactions in the last 10 minutes) and *windowed aggregates* (e.g., average spend over 24 hours vs. current spend). These behavioural markers are often stronger predictors than raw amount or location.
- **Dynamic Thresholding:** A static decision threshold (e.g., $p > 0.5$) is suboptimal. We recommend segment-based thresholding:
 - *High-Risk Segments (International/High Value):* strict thresholds to prevent large losses.
 - *Low-Risk Segments (Micropayments):* lenient thresholds to minimise customer friction.
- **Cascading Model Architecture:** Deploy a two-stage system:
 1. A fast, high-recall model (e.g., LightGBM) to screen all transactions in real-time ($< 10\text{ms}$).
 2. A slower, high-precision model (e.g., Neural Network or SVM) to review only the flagged suspicious cases.
- **Unsupervised Anomaly Detection:** Integrate unsupervised models alongside the supervised classifiers. These can detect "zero-day" fraud patterns that the supervised models have never encountered during training.
- **Continuous Monitoring & Retraining** Fraud detection is not a one-time modelling task but a dynamic system that must evolve alongside consumer behaviour.

7 Conclusion

Table 9: Final Model Performance Summary

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC	PR-AUC
Logistic Reg (Tuned)	0.956366	0.063579	0.750583	0.117227	0.828361	0.136707
Decision Tree	0.998175	0.801922	0.700233	0.747636	0.970564	0.780480
XGBoost	0.997918	0.821615	0.588345	0.685683	0.994155	0.753072
Random Forest (Tuned)	0.997592	0.937161	0.403263	0.563885	0.970314	0.726831
SVM (RBF)	0.996786	0.704675	0.288112	0.409001	0.878311	0.425281
LightGBM (Tuned)	0.997132	0.605032	0.739860	0.665688	0.983037	0.740344

In this project, we wanted to detect fraudulent credit card transactions using a range of machine learning models, from simple classifiers to advanced methods that we have studied in class. Our results highlight how different algorithms respond to the challenges of highly imbalanced fraud data.

After comparing models using metrics that matter most for rare-event detection: Precision–Recall AUC, Recall, and F1 score, the **Decision Tree classifier** is seen as the best-performing model overall.

It achieved the strongest balance between identifying fraud (recall), avoiding false alarms (precision), and maintaining consistent decision-making (F1 score). Its high PR-AUC make it especially useful for fraud analysis, where model transparency is essential for financial institutions.

Although the Decision Tree performed best, the boosting models: LightGBM and XGBoost also demonstrated strong fraud detection capabilities. LightGBM achieved recall levels comparable to the Decision Tree while offering greater scalability for deployment on large datasets. XGBoost provided great precision and the highest ROC-AUC among all models. These models serve as powerful alternatives when operational constraints prioritize speed or when complex nonlinear patterns must be captured effectively. Models like Logistic Regression and SVM are observed to be not that effective, due to the nonlinear and complex nature of fraud patterns. Nevertheless, they still provided useful baselines for comparisons and demonstrated how model choice affects the recall–precision trade-off.

8 Adherence to Submission Guidelines

We confirm that we have followed the submission instructions, the README completeness, data access, code execution instructions, file organization, and naming conventions.

9 Individual Contributions

Dhwaj Jain :

- Data Preprocessing
- Logistical Regression
- Decision Tree Classifier
- XGBoost
- Random Forest Classifier
- Evaluation Metrics
- Report Writing

- README file

Maanya Raichura:

- EDA and Advance EDA
- SVM
- LightGBM
- SHAP & LIME
- Hyperparameter Tuning (RandomizedSearchCV)
- Evaluation Metrics
- Report Writing

References

- [1] GeeksforGeeks. *SMOTE for Imbalanced Classification with Python*. <https://www.geeksforgeeks.org/machine-learning/smote-for-imbalanced-classification-with-python>. Accessed: 2025-10-26. May 2024.
- [2] Kartik Shenoy. *Credit Card Transactions Fraud Detection Dataset*. <https://www.kaggle.com/datasets/kartik2112/fraud-detection/code>. Accessed: 2025-10-26. 2019.
- [3] GeeksforGeeks. *XGBoost*. <https://www.geeksforgeeks.org/machine-learning/xgboost/>. Accessed: 2025-10-26. Sept. 2021.
- [4] xgboost developers. *XGBoost Documentation – Xgboost 1.5.1 Documentation*. <https://xgboost.readthedocs.io/en/stable/>. Accessed: 2025-10-26. 2022.
- [5] Tutorialspoint. *Machine Learning - Performance Metrics*. https://www.tutorialspoint.com/machine_learning/machine_learning_performance_metrics.htm. Accessed: 2025-10-26.
- [6] Edureka. *Data Science Project - Credit Card Fraud Detection Using Machine Learning*. <https://www.youtube.com/watch?v=PmssN0Aeqdk>. Accessed: 2023-09-12. Apr. 2020.
- [7] GeeksforGeeks. *Steps to Build a Machine Learning Model*. <https://www.geeksforgeeks.org/machine-learning/steps-to-build-a-machine-learning-model/>. Accessed: 2025-10-26. Feb. 2024.
- [8] GeeksforGeeks. *Advanced EDA*. <https://www.geeksforgeeks.org/data-science/advanced-eda/>. Accessed: 2025-12-13. Aug. 2025.
- [9] GeeksforGeeks. *Comparing Randomized Search and Grid Search for Hyperparameter Estimation in Scikit Learn*. <https://www.geeksforgeeks.org/machine-learning/comparing-randomized-search-and-grid-search-for-hyperparameter-estimation-in-scikit-learn/>. Accessed: 2025-12-13. Dec. 2022.
- [10] GeeksforGeeks. *LightGBM (Light Gradient Boosting Machine)*. <https://www.geeksforgeeks.org/machine-learning/lightgbm-light-gradient-boosting-machine/>. Accessed: 2025-12-13. July 2020.
- [11] GeeksforGeeks. *Logistic Regression in Machine Learning*. <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>. Accessed: 2025-12-13. May 2017.

- [12] GeeksforGeeks. *Random Forest Regression in Python*. <https://www.geeksforgeeks.org/machine-learning/random-forest-regression-in-python/>. Accessed: 2025-12-13. June 2019.
- [13] GeeksforGeeks. *RBF SVM Parameters in Scikit Learn*. <https://www.geeksforgeeks.org/python/rbf-svm-parameters-in-scikit-learn/>. Accessed: 2025-12-13. July 2023.
- [14] IBM. *What Is a Decision Tree?* <https://www.ibm.com/think/topics/decision-trees>. Accessed: 2025-12-13. Nov. 2021.
- [15] Zoumana Keita. *Explainable AI - Understanding and Trusting Machine Learning Models*. <https://www.datacamp.com/tutorial/explainable-ai-understanding-and-trusting-machine-learning-models>. Accessed: 2025-12-13. May 2023.
- [16] John D. Kelleher, Brian Mac Namee, and Aoife D’Arcy. *Fundamentals of Machine Learning for Predictive Data Analytics*. Cambridge, Massachusetts: MIT Press, 2015.
- [17] scikit-learn developers. *Decision Trees — Scikit-Learn Documentation*. <https://scikit-learn.org/stable/modules/tree.html>. Accessed: 2025-12-13. 2025.
- [18] scikit-learn developers. *sklearn.model_selection.RandomizedSearchCV|Scikit-Learn Documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html. Accessed: 2025-12-13. 2019.
- [19] Adam Shafi. *Sklearn Random Forest Classifiers in Python Tutorial*. <https://www.datacamp.com/tutorial/random-forests-classifier-python>. Accessed: 2025-12-13. Feb. 2023.
- [20] Sidharth. *The RBF Kernel in SVM: A Complete Guide*. <https://www.quarkml.com/2022/10/the-rbf-kernel-in-svm-complete-guide.html>. Accessed: 2025-12-13. Oct. 2022.